

SPBANKLENS 360

Enterprise Banking Operations & Risk Intelligence Platform

TECHNICAL ARCHITECTURE DOCUMENT

Version 1.0 Prototype

Conceptualized & Developed by
Sagarika Paul

© 2026 Sagarika Paul. All Rights Reserved.

Document Information

Field	Value
Title	Technical Architecture Document (TAD)
Application	SPBANKLENS 360
Subtitle	Enterprise Banking Operations & Risk Intelligence Platform
Version	1.0 Prototype
Date	June 2026
Developer	Sagarika Paul
Audience	Architects, Engineers, Reviewers, Auditors
Classification	Internal Use

Version History

Version	Date	Author	Description
0.1	May 2026	Sagarika Paul	Initial outline
0.9	Jun 2026	Sagarika Paul	Review draft
1.0	Jun 2026	Sagarika Paul	First release — documents implemented prototype

Table of Contents

1. Executive Summary	5
2. System Architecture	6
3. Technology Stack	8
4. Application Architecture	10
5. Authentication & Authorization	13
6. Dashboard Architecture	15
7. Account 360 Architecture	17
8. Knowledge Notebook Architecture	18
9. Rule-Based Risk Engine	20
10. Workflow Engine	22
11. Data Upload & Integration	23
12. Reporting Architecture	24
13. Security Architecture	25
14. Error Handling	27
15. Audit Trail	28
16. Performance Considerations	29
17. Deployment Architecture	30
18. Future Architecture Roadmap	31
19. Technical Appendix	33

1. Executive Summary

SPBANKLENS 360 is a Risk Intelligence and Operations layer for banks, implemented as a modern single-page web application backed by a TanStack Start full-stack React framework on an edge runtime. It consolidates portfolio, risk, recovery, legal, inspection and compliance information into a single role-aware operational view, and is deliberately positioned as a READ-ONLY platform that does not post transactions to the Core Banking System (CBS) or Loan Origination System (LOS).

The architecture is built around four guiding principles: (1) Read-Only Intelligence — never source-of-truth; (2) Role-Based Operational Views — distinct dashboards for Branch, Regional Office, Central Office and Administrator; (3) Operational Memory — an immutable, versioned Knowledge Notebook capturing institutional knowledge; and (4) Explainable Risk — every risk score traceable to the rules that produced it.

NOTE This document describes the implemented prototype. Items requiring production-grade implementation (persistent database, enterprise SSO, broker-based integration, etc.) are clearly identified in Section 18 — Future Architecture Roadmap, and are NOT part of v1.0.

2. System Architecture

SPBANKLENS 360 v1.0 implements a single-page application with server-side rendering (SSR) capability provided by TanStack Start. The runtime is split between client-side rendering for the role dashboards and operational modules, and server functions running on an edge serverless runtime for authenticated session validation.

2.1 Frontend Architecture

The frontend is a React 19 application bootstrapped through TanStack Start v1 with Vite 7 as the build tool. File-based routing under `src/routes/` generates a type-safe route tree consumed by the TanStack Router. UI composition uses Radix primitives wrapped by shadcn-style components in `src/components/ui/`, and Tailwind CSS v4 supplies the design tokens through `src/styles.css`.

2.2 Backend / Mock Backend

The prototype intentionally does not use a persistent database. Operational state — Knowledge Notebook entries, workflow tasks, risk engine outputs, escalations, audit metadata — is held in in-memory module stores located in `src/components/*/store.ts` files (e.g., `workflow/store.ts`, `knowledge/store.ts`, `central/risk-intel-store.ts`, `phase8/data.ts`). Mock data engines under `src/components/branch/mock-data.ts` and `src/components/phase8/data.ts` provide realistic demo portfolios, and a Demo/Upload mode toggle allows replacement with user-uploaded Excel/CSV.

Server-side logic is limited to authentication: a TanStack Start server function validates credentials and issues an encrypted `httpOnly` session cookie. All other server-side capabilities (transactions, scheduled jobs, persistence) are scoped to future architecture.

2.3 Authentication Approach

Authentication is implemented via server-side encrypted `httpOnly` cookies issued by a TanStack Start `createServerFn` handler. Role information is encoded into the session and validated on each protected route. A demonstration credentials panel is shown on the login screen for prototype evaluation.

2.4 Data Storage

Implemented: in-memory stores per module, browser session for transient UI state. No persistent storage in v1.0. Excel/CSV uploads via the Data Integration Hub are parsed in the browser using SheetJS (xlsx) and projected into the in-memory stores.

2.5 Read-Only Operational Intelligence Layer

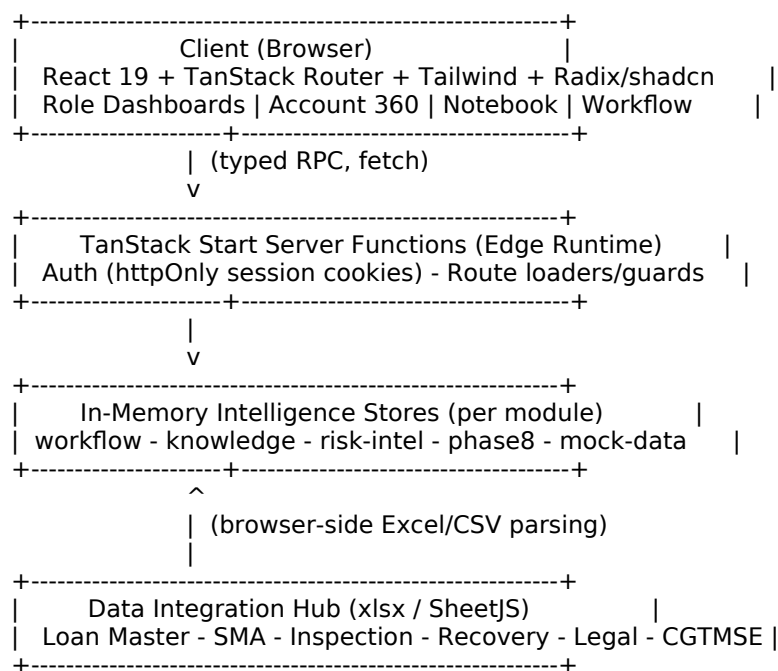
By design, the application performs no writebacks to upstream systems. All data flows are one-way: source systems → uploaded files / mock data → in-memory intelligence stores → role dashboards. This boundary is enforced architecturally — there are no client modules that POST to CBS/LOS endpoints.

2.6 Dashboard Architecture

Each role mounts a dedicated dashboard composition: BranchDashboard, RegionalDashboard, and the Central Executive Command Centre. Dashboards consume the same intelligence stores and render KPI tiles, charts (Recharts), heat maps and task widgets. Drill-down navigation uses TanStack Router's typed Link/useNavigate APIs to descend from enterprise to regional to branch to account level.

2.7 High-Level Architecture

[Diagram — Refer to SPBANKLENS_360_Architecture.png (Appendix A) for the rendered diagram]



3. Technology Stack

The stack listed below reflects the implemented prototype and is verified against package.json.

Layer	Technology	Version (approx.)	Notes
Frontend framework	React	19.x	Functional components, hooks
Meta framework	TanStack Start	1.16x	SSR + server functions on edge runtime
Routing	TanStack Router	1.17x	File-based, type-safe
Language	TypeScript	5.x	Strict mode
Build tool	Vite	7.x	Dev server and production build
Styling	Tailwind CSS	4.x	Native @theme, src/styles.css
UI primitives	Radix UI	1.x / 2.x	Wrapped by shadcn-style components
Component library	shadcn (custom)	—	src/components/ui/
State management	React state + module stores	—	No global Redux/Zustand; module-scoped stores in src/components/*/store.ts
Server state	TanStack Query	5.10x	Available; primary use is auth/session
Forms	react-hook-form + zod	7.x / —	Hookform resolvers
Charts	Recharts	2.15.x	KPI/trend/heatmap charts
Excel/CSV I/O	SheetJS (xlsx)	0.20.3	Upload parsing and exports
Date utilities	date-fns	4.x	Reminder window calculations
Icons	lucide-react	0.5x	All iconography
Notifications	sonner	2.x	Toasts
Database	None (prototype)	—	In-memory stores; see §18 for future
Auth mechanism	Server-issued httpOnly cookie	—	TanStack Start server function
Hosting	Cloudflare Workers (edge)	—	Via TanStack Start adapter

Layer	Technology	Version (approx.)	Notes
Version control	Git	—	Standard
NOTE Technologies not used in v1.0 (and therefore NOT documented as implemented): Redux, Zustand, Supabase/Postgres, REST/GraphQL gateway to CBS, OAuth/SAML SSO, message brokers, container orchestration.			

4. Application Architecture

4.1 Folder Structure

src/	
routes/	TanStack file-based routes (URL <-> filename)
__root.tsx	Root layout + global head metadata
index.tsx	Landing / role selection
branch/...	Branch portal routes
regional/...	Regional Office routes
central/...	Central Office routes (incl. risk-intel)
api/...	Server route handlers (auth)
components/	
AppShell.tsx	Sidebar + topbar + content shell
BrandFooter.tsx	Global branded footer
PrototypeDisclaimer.tsx	
ui/	shadcn-style primitives (Radix wrapped)
branch/	BranchDashboard, Account360, mock-data
regional/	Regional dashboard, heat map, comparison
central/	Executive Command Centre, risk-intel-store
knowledge/	Notebook 2.0, store, QuickNoteDialog
workflow/	Workflow Centre, store, reminder engine
phase8/	NPA, Legal, CGTMSE, Inspection, War Room
hooks/	Reusable hooks
lib/	auth.ts, brand.ts, export-brand.ts, utils
styles.css	Tailwind v4 theme tokens
router.tsx	Router bootstrap
start.ts / server.ts	TanStack Start entry
routeTree.gen.ts	GENERATED -- do not edit

4.2 Module Organization

Each functional area is encapsulated in a sub-folder under src/components/ alongside its own store (Notebook, Workflow, Risk Intel, Phase 8 modules). This co-location keeps view, state and helpers together and minimises cross-module coupling.

4.3 Component Hierarchy

```

<RootRoute>
+- <AppShell>          (Sidebar + TopBar + Outlet)
  +- <RoleLandingPage>  (/)
  +- <BranchModuleShell> (branch/*)
    | +- <BranchDashboard>
    | +- <Account360>
    | +- <KnowledgeNotebook>
    | +- <WorkflowCentre>
  +- <RegionalDashboard> (regional/*)
  +- <CentralCommandCentre>(central/*)
    +- <RiskIntelHub>
    +- <Phase8 Modules> (NPA, Legal, CGTMSE, Inspection)
    +- <OfficerContinuity>

```

4.4 Routing / Navigation

Routes are declared by filename under src/routes/ and consumed via the TanStack Router. Underscore-prefixed segments form pathless layouts (e.g., _authenticated

routes are gated to logged-in sessions). The router's loaders are used for auth guards; data fetching for in-memory stores happens inside components on mount.

4.5 Shared Components

- AppShell — the enterprise navigation shell (sidebar, topbar, search, notifications, profile).
- BrandFooter / PrototypeDisclaimer — global branding.
- BranchModuleShell — consistent sub-navigation across branch sub-modules.
- ui/* — Radix-wrapped shadcn primitives (Dialog, Sheet, Tabs, Table, Tooltip, Toast).

4.6 Reusable Services / Libraries

- lib/auth.ts — session retrieval and role guards.
- lib/brand.ts — centralised product naming.
- lib/export-brand.ts — Excel export watermarking via SheetJS.
- knowledge/store.ts, workflow/store.ts, central/risk-intel-store.ts — in-memory module stores.

[Diagram — Refer to SPBANKLENS_360_DataFlow.png (Appendix A) for the cross-module data flow]

5. Authentication & Authorization

5.1 Login Process

1. User selects a role on the Landing Page.
2. Submits credentials on the role-specific Login screen.
3. Browser POSTs to a TanStack Start server function (createServerFn).
4. Server validates credentials against the prototype credentials list.
5. On success, the server issues an encrypted httpOnly Set-Cookie with role + identity claims.
6. Client navigates to the role dashboard; subsequent navigations reuse the cookie.

5.2 Session Management

Sessions are server-side encrypted and signed; the cookie is httpOnly, Secure and SameSite. No session data is persisted in localStorage or sessionStorage — this is a deliberate departure from the earlier prototype design and was a security hardening change.

5.3 Role-Based Access Control

Authorisation is enforced at two layers: (a) route-level loaders check the session and redirect unauthenticated users to /auth; (b) component-level guards (lib/auth.ts) check the role claim and render or hide module entry points.

5.4 Security Model

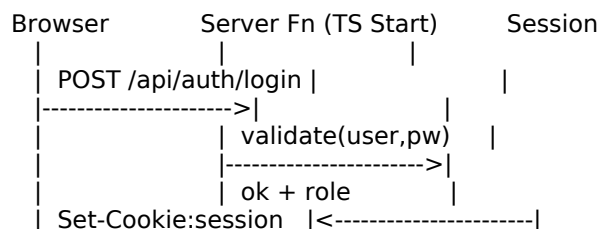
- httpOnly cookies prevent XSS-based session theft.
- Server-side encryption prevents client-side tampering.
- Role enforced on every protected loader.
- Demo credentials panel shown only on the login screen, suitable for evaluation.

5.5 Demo Authentication

For evaluation, valid role-credential pairs are provided on each login screen. In production these are replaced by the bank's enterprise identity provider (AD/LDAP/SSO).

5.6 Sequence Diagram

[Diagram — Login sequence — Browser → Login screen → Server Fn → Cookie → Role Dashboard]



```
|<-----|           |           |
| GET /<role>/dashboard|       |
|----->|           |           |
|         | verify(cookie) |       |
| 200 OK (HTML+data) |       |
|<-----|           |           |
```

6. Dashboard Architecture

6.1 Branch Dashboard

Implemented at `src/components/branch/BranchDashboard.tsx`. Shows 16 KPI tiles, a Risk Health Index gauge, portfolio trend, risk distribution charts and today's tasks. Data sourced from `src/components/branch/mock-data.ts` (Demo) or the in-memory store after Hub upload.

6.2 Regional Office Dashboard

Implemented under `src/components/regional/`. Includes Branch Risk Heat Map, branch ranking, branch comparison tool and the Follow-up & Escalation tracker. Drill-down opens the corresponding branch dashboard in read-only supervisory mode.

6.3 Central Office Dashboard

Implemented under `src/components/central/` as the Executive Command Centre with National Health Index, sector concentration, geographic exposure, governance index and ATR tracker. Risk Intelligence module (risk-intel-store) drives explainable risk views.

6.4 Data Flow

Demo data or uploaded data → in-memory stores → React selectors → KPI/chart components. Auto-refresh hooks (where configured) re-read the store at intervals to reflect new uploads and engine outputs without a full page reload.

6.5 Navigation & Drill-down

Navigation uses typed TanStack Router Links. Drill-down preserves filter context via search params so that 'click → drill → back' returns to the same filtered view.

6.6 Component Interactions

[Diagram — Component interaction map — Dashboards ↔ Stores ↔ Account 360 ↔ Notebook ↔ Workflow]

7. Account 360 Architecture

Account 360 is implemented at `src/components/branch/Account360.tsx` and forms the single drill-down target from every list view. The screen composes a header (customer, loan, risk summary) and tabbed panels for Customer, Loan, Documents, Risk, Inspection, Recovery, Legal, Knowledge Notebook, Attachments and Timeline.

7.1 Customer Data Display

CIF, demographics, contact and KYC details are read from mock data or the uploaded customer master.

7.2 Loan Information

Sanctioned, outstanding, EMI, DPD and SMA fields are computed from the Loan Master upload or mock generator.

7.3 Timeline

A chronological feed aggregating notebook entries, recovery actions, inspections, escalations and uploads for the account.

7.4 Knowledge Notebook Integration

The Knowledge Notebook tab and Quick Note dialog write into `knowledge/store.ts`, scoped to the active account ID. New versions are emitted on every save.

7.5 Integration with Dashboards

Account 360 receives the account ID via TanStack Router path params. KPI tiles and lists across dashboards link to Account 360 using typed Link with `params={{ accountId }}`.

8. Knowledge Notebook Architecture

8.1 Data Model

Field	Type	Description
id	string	Unique note identifier
accountId	string	Account context
section	enum (10 values)	Inspection, Customer Interaction, Recovery, Document, Legal, etc.
fields	object	Structured fields per section
attachments	Attachment[]	File references (prototype: in-memory)
author	string	Officer identifier
createdAt	ISO date	Creation timestamp
version	number	Monotonic per-note version
supersedes	string?	Previous version id (if any)

8.2 Version History

Notes are immutable. Saving an edited note creates a new record with an incremented version that references the previous via supersedes. The store exposes selectors for both 'latest' and 'full history' reads.

8.3 Operational Note Lifecycle

Create -> Save (v1) -> Edit -> Save (v2, supersedes v1)
 |
 +-> Officer Handover bundles latest + history

8.4 Attachments

Implemented as in-memory file references bound to a note version. Future architecture: persistent object storage (see §18).

8.5 Officer Continuity

The handover module reads all open tasks, pending follow-ups and notebook entries for the outgoing officer and bundles them into a handover pack for the incoming officer's acknowledgement.

8.6 Data Flow

UI (Account 360 / Quick Note)
 | addNote()
 v
 knowledge/store.ts ---> versioned records (in-memory)
 | selectors
 v

Notebook panel - Timeline - Handover pack

9. Rule-Based Risk Engine

9.1 Inputs

- Loan attributes — DPD, outstanding, SMA bucket, sanctioned amount.
- Account attributes — sector, vintage, security coverage.
- Operational signals — overdue tasks, missed inspections, expired documents.

9.2 Rules

13 weighted rules implemented in `src/components/central/risk-intel-store.ts`. Each rule exposes `id`, `label`, `weight`, `predicate(account)` and `contribution(account)`. Examples include DPD bands, security shortfall, document expiry, missed inspection and recovery inactivity.

9.3 Risk Scoring Process

```
for each account:
  score = 0
  contributions = []
  for each rule in 13 rules:
    if rule.predicate(account):
      score += rule.weight * rule.contribution(account)
      contributions.push({ rule, value })
  band = bucket(score) // Green / Amber / Red
  emit { score, band, contributions }
```

9.4 Explainability

The Explainable Risk panel on Account 360 renders the contributions array sorted by impact, naming every rule that fired and the value it added. This makes every score traceable to its inputs.

9.5 Outputs

- Per-account score and band (Green/Amber/Red).
- EWS triggers feeding the Task Generation Engine.
- Portfolio-level distributions consumed by dashboards.

9.6 Configuration

Thresholds and rule weights are exposed in the Admin module. In v1.0, edits persist for the current session only (in-memory).

[Diagram — Risk Engine flowchart]

10. Workflow Engine

10.1 Task Lifecycle

Created --> Assigned --> In Progress --> Completed --> Closed
|
+--> (SLA breach) --> Escalated

10.2 Assignment

Tasks are created automatically by the Task Generation Engine in response to EWS triggers, or manually by officers/managers. Assignment uses the role pool defined in the Escalation Matrix.

10.3 Escalation

Tasks unactioned within their SLA escalate per the matrix (Officer → Manager → Regional → Central). Escalations are surfaced on dashboards and the Follow-up Tracker.

10.4 Reminder Logic

Implemented in src/components/workflow/store.ts. A window-based reminder engine evaluates due-date bands (Due Today, Overdue, Upcoming) on read and emits notification entries; the Email Automation module simulates outbound email.

10.5 Completion Process

Officer marks Completed; Manager Workbench reviews and Closes. Closed tasks are immutable and audit-trailed.

11. Data Upload & Integration

11.1 Excel / CSV Upload

The Data Integration Hub accepts .xlsx and .csv via the browser. Parsing uses SheetJS (xlsx 0.20.3) and is performed entirely client-side; no file leaves the browser.

11.2 Validation

- Required columns present.
- Data types and date formats enforced.
- Reference integrity (e.g., branch code in master).
- Row-level error report displayed; user corrects and re-uploads.

11.3 Data Mapping

Column headers are mapped to internal model fields via a per-template map. Templates: Loan Master, SMA, Inspection, Recovery, Legal, CGTMSE.

11.4 Error Handling

Validation errors are aggregated into a results panel; the user can download a corrections file. Processing only commits on a clean validation.

11.5 Processing Flow

Upload -> Parse (xlsx) -> Validate -> Map -> Commit to in-memory store
 | |
 +--> error report +--> risk engine re-run

12. Reporting Architecture

Reports are rendered as React tables/charts and exported to Excel via SheetJS. Exports are watermarked through lib/export-brand.ts with the SPBANKLENS 360 product mark and developer credit.

12.1 Generation

Each module exposes report templates relevant to its role scope (Branch, Regional, Central).

12.2 Filtering

Common filters: region, branch, product, sector, period, risk band. Filters update reactive selectors over the in-memory store.

12.3 Export

xlsx.utils builds the workbook; lib/export-brand.ts injects the branded header sheet. Output is a .xlsx file streamed to the browser.

12.4 Dashboard Summaries

KPI tiles on each dashboard are themselves summary reports; clicking the tile opens the underlying account list for the same filter.

13. Security Architecture

13.1 Authentication

Server-issued encrypted httpOnly cookie via TanStack Start server function; no client-stored tokens.

13.2 Authorization

Role claim on session; route loaders + component guards enforce role boundaries.

13.3 Input Validation

Form inputs are validated via react-hook-form + zod schemas. Upload inputs validated by the Hub validator (see §11).

13.4 Audit Logging

Notebook saves, task transitions, escalations, configuration changes and metadata edits are recorded in the in-memory audit trail (see §15).

13.5 Session Handling

Cookies are httpOnly, Secure, SameSite. Inactive sessions expire.

13.6 Data Protection

All processing in v1.0 is client-side after upload; no network egress of uploaded files. Future production deployment will require server-side at-rest encryption and KMS-managed keys (see §18).

13.7 Role Isolation

Branch users cannot see other branches; Regional users see only their region; Central users see enterprise scope. Admins do not have operational task scope.

13.8 Prototype Limitations vs Production Recommendations

Concern	Prototype (v1.0)	Production Recommendation
Identity	Demo credentials	Enterprise SSO (AD/LDAP/SAML/OIDC)
Storage	In-memory	Encrypted RDBMS + object storage
Audit	In-memory, session-scoped	WORM/append-only audit store
Transport	TLS on edge	TLS + mTLS for integrations
Secrets	Not required (no integrations)	KMS-managed secrets
Pentest	Not performed	Annual + on major release

Concern	Prototype (v1.0)	Production Recommendation
LIMITATION v1.0 is an evaluation prototype. It is NOT certified for production use with live customer data.		

14. Error Handling

14.1 Validation Errors

Form-level validation messages via react-hook-form + zod; upload-level error reports via the Data Integration Hub.

14.2 Upload Failures

Parse and validation errors are caught client-side and rendered as an actionable error panel; commit is blocked until clean.

14.3 Authentication Failures

Bad credentials return a user-friendly message; session validation failures redirect to /auth.

14.4 User-Friendly Messages

All caught errors surface via sonner toasts and in-page banners. Stack traces are never shown to end users.

14.5 Logging

Prototype: console + in-memory audit trail. Production: centralised log sink (see §18).

15. Audit Trail

15.1 Logged Events

- User login / logout.
- Notebook entry create / new version.
- Task create / update / complete / close / escalate.
- Upload commits and rejections.
- Configuration changes (thresholds, SLAs).
- Officer handover pack generation and acknowledgement.

15.2 Version History

Notebook and configuration changes retain full version history with author, timestamp and superseded-by reference.

15.3 User Activity

Per-user activity feed available to the Administrator in the prototype's Admin Console.

15.4 Change Tracking

Metadata edits record before/after values. The audit trail is read-only — no API allows redaction in v1.0.

LIMITATION Audit data in v1.0 is in-memory and session-scoped. Production deployment requires a persistent, append-only audit store.

16. Performance Considerations

16.1 Expected Performance

As a single-page React application on a modern browser, screen transitions are near-instant. Dashboard rendering with mock data of typical branch volumes (a few hundred accounts) is sub-second on a mid-range laptop.

16.2 Large Dataset Handling

For uploads with tens of thousands of rows, SheetJS parsing remains the main cost. UI grids use virtualization in the larger views (Workflow Queue, Account Lists). Heat maps aggregate before rendering.

16.3 Dashboard Responsiveness

Charts are memoised on the underlying selector output; auto-refresh windows are configurable to avoid unnecessary re-renders.

16.4 Scalability

The architecture is designed to scale horizontally on the edge runtime. The persistent data tier required for true scale is a future deliverable (see §18) — the prototype itself does not persist data.

NOTE Stated performance characteristics describe the prototype with mock and uploaded data. Production scale numbers must be re-baselined after the persistent backend is implemented.

17. Deployment Architecture

17.1 Hosting Platform

The application is hosted on Lovable's managed Cloudflare Workers edge runtime via the TanStack Start adapter. Static assets are served from the edge cache; server functions execute as Workers.

17.2 Build Process

vite build produces the SSR bundle plus client assets. vite build --mode development is used for preview builds. The TanStack Router plugin regenerates src/routeTree.gen.ts on every build/dev run.

17.3 Environment Configuration

v1.0 has no required runtime environment variables (no external integrations). Future versions will introduce SSO and database secrets managed by the deployment platform.

17.4 Deployment Workflow

Code change -> Vite build -> Lovable deploy -> Edge Worker live
+- Preview URL
+- Published URL

18. Future Architecture Roadmap

FUTURE Items in this section are NOT implemented in v1.0. They are forward-looking recommendations for a production deployment.

18.1 Enterprise Authentication

Integrate with the bank's identity provider via SAML 2.0 or OIDC. Replace demo credentials. Add MFA and conditional access.

18.2 Secure API Integration

Introduce a server-side integration broker that pulls from CBS/LOS/legal/inspection systems on scheduled jobs, with mTLS, signed payloads and idempotent writes to the intelligence store.

18.3 Production Database

Migrate in-memory stores to a managed relational database (Postgres) with row-level security; object storage for notebook attachments; an append-only audit store (WORM).

18.4 Mobile Application

React Native or PWA companion focused on inspections, recovery visits and notebook capture in the field.

18.5 High Availability

Multi-region edge + active-active database with managed failover; SLO of 99.9% for read paths.

18.6 Disaster Recovery

Point-in-time recovery on the database; cross-region backups; documented RTO/RPO targets and tabletop exercises.

18.7 AI-assisted Analytics (with human oversight)

LLM-assisted summarisation of notebook entries and inspection reports, anomaly detection over the time-series KPIs, and natural-language query over the intelligence layer — all gated by human review before action.

19. Technical Appendix

19.1 Architecture Diagrams

See SPBANKLENS_360_Architecture.png in the documentation set.

19.2 Data Flow Diagrams

See SPBANKLENS_360_DataFlow.png in the documentation set.

19.3 Sequence Diagrams

See §5.6 (Login Sequence).

19.4 Component Diagrams

See §4.3 (Component Hierarchy).

19.5 Entity Relationship Diagram

NOTE Not applicable in v1.0 — the prototype does not use a relational database. An ERD will be authored alongside the production data model (see §18.3).

19.6 Folder Structure

See §4.1 for the canonical layout.

19.7 Configuration Notes

- vite.config.ts — TanStack Router plugin + Tailwind v4 plugin.
- src/styles.css — Tailwind @theme tokens (Banking Navy palette).
- src/router.tsx — router bootstrap with default error/notFound components.
- src/start.ts — TanStack Start client function middleware (auth attacher).
- No runtime environment variables required in v1.0.

NOTE Implementation Detail Not Available: production-grade deployment topology (LB/CDN policy, DR runbook, monitoring SLOs) is out of scope for the prototype and will be authored as part of the production rollout package.